
Proyecto Grupal 2

Simulación de procesadores básicos

Fecha de asignación:	21 abril 2026
Grupos:	3-4 personas

Fecha de entrega:	21 mayo 2026
Profesores:	Luis Chavarría Zamora

Mediante el desarrollo de este proyecto, los estudiantes revisarán los diferentes conceptos básicos de la arquitectura segmentada.

1. Especificación

Se desarrollará un sistema completo que simula el comportamiento de un procesador segmentado (el sistema debe ser desarrollado por los estudiantes, no es válido usar para la presentación una herramienta ya implementada como por ejemplo Ripes u otro). Este procesador tendrá las siguientes características:

1. **El sistema completo se debe ejecutar en un lenguaje de alto nivel (como Python por ejemplo).**
2. **El código debe funcionar con la sintaxis de RISC-V.** Es decir, el mismo código suyo debe funcionar en cualquier otra interfaz RISC-V. Consejo: use las herramientas de la tarea 2 para confirmar.
3. Definir el conjunto de instrucciones (al menos 10 instrucciones comunes: carga, almacenamiento, aritméticas y de salto).
4. Mostrar gráficamente los módulos de hardware y cuándo se están utilizando según el modelo escogido.
5. Mostrar la resolución de riesgos, el procesador tendrá dos versiones ejecutándose simultáneamente, debe seleccionar entre estas implementaciones (deben estar las cuatro realizadas para escoger):
 - a) Procesador uniclo.
 - b) Procesador multiciclo.
 - c) Procesador segmentado con riesgos y solucionando con stalls.
 - d) Procesador segmentado con unidad de riesgos y adelantamiento.
6. Para el segmentado: Mostrar la instrucción que se ejecuta en las cinco etapas del Pipeline (IF, ID, EX, MEM, WB).

7. Deben mostrar al menos tres métricas de la ejecución (de cada una debe haber dos columnas pues se ejecutan dos al mismo tiempo). Se mostrará un historial de las últimas 10 ejecuciones (20 en realidad).
8. Deben tener un conjunto de instrucciones donde se demuestre el funcionamiento del procesador. **Muestre ejemplos donde se solucione uno y en el otro no, tenga lista una explicación.**
9. Habrá tres modos de funcionamiento temporales:
 - a) Un ciclo de ejecución o *step by step*. Vean que un ciclo es diferente según el procesador.
 - b) Ejecución del programa a un cierto ritmo (un ciclo cada unidad de tiempo).
 - c) Ejecución completa al final.
 - d) Para el segmentado: Escoger eventos donde aparezcan posibles *Hazard*. Por ejemplo: que se detenga hasta que encuentre un riesgo de control.
10. Los estudiantes le asignarán una latencia 'realista', basado en la duración de cada elemento de hardware. Debe referenciarse al estado del arte en la defensa. Gracias a esto el ciclo puede durar más con riesgo contra el que no tiene.
11. Deben existir los siguientes elementos visibles al usuario:
 - a) Ciclo de ejecución.
 - b) Tiempo desde que inició el programa con su primera instrucción.
 - c) Valor del PC.
 - d) Valor de los registros.
 - e) Contenido en memoria desde 0 hasta la última celda como el vector resultante, indicando el valor de cada celda.
 - f) Ubicación de cada instrucción en el pipeline.

2. Metodología de trabajo

El proyecto debe seguir los siguientes aspectos de desarrollo colaborativo en git, sino, la parte funcional no será calificada y obtendrá nota de cero:

1. Utilice una cuenta de repositorio gratuita.
2. Cree un repositorio con el siguiente nombre: `<user_id>_compu_archi_found_2G1_2026`.

3. El `user_id` estará compuesto por la primera letra del nombre y el apellido de quien haga el repositorio.

Se usará un repositorio por grupo, el generador le realiza la invitación a las demás personas.

4. El repositorio debe ser privado. Proporcione acceso a `compu-archi-found-1S2026` (GitHub).
5. El repositorio de Git contendrá dos ramas principales: `master` y `development`.
6. Inicialmente, la rama de `development` se crea a partir del `master`.
7. Al trabajar en un proyecto, los estudiantes deben crear una nueva rama de trabajo desde `develop` y cuando la función esté lista, la rama debe fusionarse para `develop`. Cualquier corrección o modificación adicional después de `merge` debería requerir que se repita el proceso (es decir, crear la rama desde `develop` y fusionar los cambios más tarde). Una vez que el código de desarrollo esté listo, se fusionará con `master` y se debe crear una `tag`. El proceso se describe en la siguiente Figura 1.

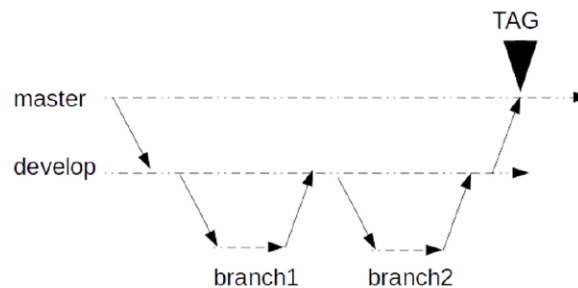


Figura 1: *Git workflow*

Adicionalmente se coloca este [enlace recomendado](#).

8. Después de haber realizado algunos proyectos la rama `master` debe verse así:

- `master/`
 - `proyecto_1`
 - `proyecto_2`
 - ...
- ...

Donde cada directorio de `proyecto_x` contiene todos los entregables para cada proyecto.

No es permitido realizar todo el trabajo en un solo commit, es decir, que realice el trabajo de forma local y solo suba el último entregable en el repositorio. Si no, obtendrá nota de cero. Debe mostrar avance incremental durante todas las semanas (se revisarán estadísticas). Se revisará el trabajo en equipo.

3. Evaluación y entregables

Se programarán horas de defensa para el mismo día de la entrega y la agenda estará disponible según la disponibilidad del profesor. Todos los archivos (incluyendo código fuente) serán entregados a las 11:50 pm ese mismo día (**realícenlo progresivamente y no lo dejen para el final**). **Si algo no queda claro o no sabe, no lo asuma, pregúntele al profesor.** Como recomendación se presenta el cronograma de trabajo de la Tabla 1.

Tabla 1: Cronograma sugerido para el proyecto

Semana	Actividades sugeridas
1	Estudio del problema y mockup del sistema a construir (desde el punto de vista de ventanas). Consignación en bitácora. Se sugiere validar con el profesor.
2	Esquematización de la solución a nivel de software. Consignación en bitácora.
3	Simulación y pruebas iniciales del algoritmo. Consignación en bitácora.
4	Integración de las soluciones y verificación del algoritmo. Consignación en bitácora.
5	Validación de las soluciones. Consignación en bitácora.

La evaluación del proyecto se da bajo los siguientes rubros contra rúbrica correspondiente:

- Presentación proyecto 100 % funcional (70 %): La defensa se realizará de **forma presencial** en un horario a definir. La defensa se debe realizar en un espacio de **media hora** donde se analizará el funcionamiento total del sistema de forma minuciosa. Todos los integrantes deben intervenir en la defensa. Adicionalmente se deben mostrar simulaciones según lo solicite el profesor.

Los entregables adicionales que se revisarán **durante** la defensa son los siguientes:

1. Diagrama general de todo el sistema, desde el usuario hasta los módulos.
 2. Mockups.
 3. Código fuente.
- Bitácora (Atributo evaluado: Conocimiento de Ingeniería) (15 %): Es un documento donde los estudiantes indica detalladamente los procedimientos que implementaron en el diseño,

tablas, ecuaciones booleanas, herramientas de automatización utilizadas y otras relacionadas. Debe existir una división clara de días en que se realizó. Los estudiantes deben desarrollarlo en [Overleaf](#)¹ y compartir con solo lectura al profesor del curso. El documento debe crecer en contenido conforme avanza el tiempo, no anotar las actividades al final, y no anotarlo a más tardar de dos días de haber realizado la actividad.

- Propuesta de investigación (Atributo evaluado: Investigación) (15 %): Este documento contará con las siguientes secciones donde se explique el proyecto en su totalidad:
 1. Identificación del problema complejo de ingeniería a investigar.
 2. Diseño de una propuesta de investigación.
 3. Ejecución de la metodología del plan de investigación para la obtención de datos relevantes.
 4. Análisis los datos obtenidos en el desarrollo de la investigación.
 5. Elaboración de conclusiones a partir de la síntesis y análisis de los resultados de la investigación.

Se seguirán los siguientes lineamientos:

1. Los documentos serán sometidos a control de plagios para eliminar cualquier intento de plagio con trabajos de semestres anteriores, actual o copias textuales, tendrán nota de cero los datos detectados. Se prohíbe el uso de referencias hacia sitios no confiables.
2. No coloque código fuente en los documentos, quita espacio y aporta poco. Mejor explique el código, páselo a pseudocódigo o use un diagrama.

¹Se solicita Overleaf pues cuenta con historial sencillo de seguir y es colaborativo.